

PACE-Seg: Hardware-Cost-Conditioned Elastic Semantic Segmentation with Calibrated Risk Routing for Edge Accelerators

Dũng Nguyễn

Abstract

Edge semantic segmentation faces two coupled uncertainties: visual difficulty changes under adverse conditions, and model cost changes across accelerator architectures. We present PACE-Seg, a compiler-safe elastic model with four statically deployable capacities and a runtime policy that combines measured route latency with candidate-specific error estimates from one low-cost entropy probe. Experiments use Cityscapes, four ACDC conditions, and four edge devices. The capacities span $110.2\times$ in FLOPs but only $10.3\text{--}19.5\times$ in measured candidate latency, so a proportional FLOPs proxy fails to reproduce measured budget choices. The largest elastic capacity remains within 1.1 mIoU points of a similarly sized independently trained Fast-SCNN on every evaluated split. In deployment-matched held-out replay with complete warm-route costs on TensorRT/CUDA and Hailo-8, the budget-constrained policy outperforms or matches rank escalation in 70 of 74 fair operating cells, with a macro gain of 0.0284 mIoU; it produces no budget-violating cell, compared with 46 of 120 for rank escalation. Pooled calibration gives a similar result, indicating that the gain does not require condition-specific calibrator selection. These cells are correlated evaluations, and the two back-ends share segmentation predictions. A secondary fake-quant study keeps worst-condition degradation within 1.5 mIoU points for every tested run and

elasticity level but does not establish compiled INT8 accuracy.

Keywords: semantic segmentation, elastic neural networks, dynamic inference, uncertainty calibration, edge acceleration, hardware-aware deployment

1. Introduction

Edge semantic segmentation faces two coupled uncertainties. Fog, night, rain, and snow change visual difficulty, while heterogeneous accelerators assign different costs to the same model. Static networks expose one operating point; multiple independent networks add choices but not an image-wise selection rule. Elastic networks share weights across capacities, yet elasticity alone does not identify the best candidate for an image or enforce a measured deployment budget.

PACE-Seg tests whether these gaps can be closed jointly. One compiler-safe elastic model provides four static capacities. A tiny prediction supplies an image-level entropy feature, candidate-specific calibrators estimate expected error, and measured complete-route costs define the hardware-feasible set.

The experiments answer three research questions. RQ1 asks whether measured hardware cost is necessary for candidate selection. RQ2 asks whether shared elastic training preserves segmentation quality. RQ3 asks whether candidate-specific visual-risk routing improves the quality–latency trade-off under a measured budget. Quantization-aware training is retained as a secondary deployment study rather than a primary contribution.

The contributions are as follows:

1. A compiler-safe elastic segmentation model exposes four width–depth–resolution capacities as separately compilable static graphs from one

shared training process.

2. A measured-cost candidate set replaces FLOPs with observed deployment latency when constructing hardware-feasible choices.
3. A candidate-specific calibrated-risk policy maps one inexpensive visual probe to candidate error estimates and selects capacity under an explicit hardware budget.

Table 1 and Fig. 1 summarize the study setting. Section 4 gives the full protocol.

Table 1: Study setting at a glance. Candidate-only latency is measured on four devices; complete routed latency, including probing and decision overhead, is measured on the two primary accelerator backends.

Component	Evaluated setting
Visual domains	Cityscapes clean and ACDC fog, night, rain, and snow
Prediction task	Semantic segmentation over the 19 Cityscapes train classes
Elastic choices	Tiny, small, medium, and large static capacities
Candidate-cost devices	E1, E2, E3, and E5
Complete-route backends	E1 Hailo-8 and E3 TensorRT/CUDA
Router protocol	Alternating fit/held-out validation halves; three training runs

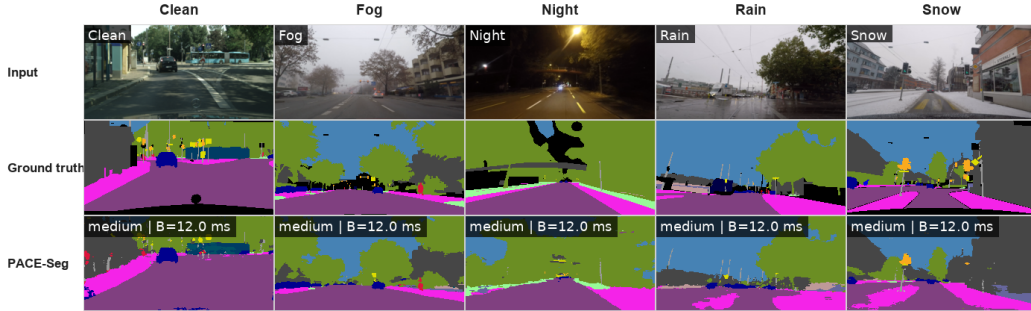


Figure 1: Visual domains and routed outputs. Columns show clean Cityscapes and ACDC fog, night, rain, and snow; rows show the input, 19-class ground truth, and configured policy-D output. Each image is the deterministic held-out median-error representative for its split. All five select medium at the displayed E3 operating point, so the panel introduces the task and visual shift rather than comparing routing policies.

The same label space appears under substantially different visibility and illumination. The fixed operating point connects these inputs to routed segmentation without implying that medium is generally optimal. Hardware feasibility is evaluated separately from visual risk using measured route tables. The 120 reported operating cells reuse images, predictions, and thresholds and are therefore descriptive correlated evaluations, not independent replications.

2. Related work

2.1. Efficient and real-time semantic segmentation

BiSeNet, STDC, PIDNet, SegFormer, and OffSeg improve the static accuracy–efficiency trade-off through different treatments of spatial detail, context, boundaries, attention, and offsets [1, 2, 3, 4, 5]. A static model still commits every image to one operating point. PACE-Seg instead studies selection among several capacities from one elastic model.

2.2. Elastic, slimmable, and once-for-all networks

Slimmable networks, US-Net, and Once-for-All share parameters across widths or broader depth–width–resolution spaces [6, 7, 8]. SlimSeg and MESS adapt this train-once principle to segmentation through adjustable widths or exits [9, 10]. These methods establish candidate sharing, but not candidate-specific per-image risk selection under a measured complete-route budget.

2.3. Hardware-aware deployment

MnasNet established direct latency as a hardware-aware search objective [13]. FasterSeg, AutoSegEdge, multi-objective edge NAS, RealtimeSeg, and HARD likewise incorporate platform cost into static segmentation design [14, 15, 16, 17, 18]. PACE-Seg addresses the complementary runtime problem: selecting among fixed candidates per image. Its RQ1 claim is limited to the failure of a proportional FLOPs proxy in the evaluated candidate–device set, not learned latency prediction in general.

2.4. Uncertainty and adaptive inference under adverse conditions

Dynamic Routing, Anytime Dense Prediction, and MESS allocate segmentation computation according to the input [11, 12, 10]. Calibration remains difficult because segmentation confidence changes with capacity, crop size, correctness, and domain shift [19, 20]; ACDC supplies the adverse-condition setting used here [21]. PACE-Seg maps one tiny-probe entropy score to a separate monotonic error estimate for each candidate, using either condition-specific or pooled calibrators. These estimates do not provide formal risk control.

2.5. Quantization of elastic networks

AdaBits and EQ-Net jointly optimize weight sharing and quantization across precision settings [24, 23]. Our fake-quant study is secondary and is

not evidence of compiled INT8 accuracy.

2.6. Positioning of PACE-Seg

Table 2 separates candidate construction, selection time, cost, and risk among the closest directions.

Table 2: Mechanism-level positioning relative to representative primary work. Entries summarize the stated operating mechanism rather than rank methods or claim exhaustive coverage. “Complete route” includes the probe, risk computation, policy decision, backend dispatch or activation, and any additional candidate inference.

Method	Candidates	Selection	Cost	Risk	Budget; deployed form
Once-for-All [8]	Elastic subnetworks	Design	Latency predictor	None	Design constraint; static subnet
SlimSeg [9]	Slimmable widths	Configured	Efficiency setting	None	No hard measured budget; static width
MESS [10]	Multi-exit network	Per image	Exit latency	Exit confidence	No complete-route constraint; multi-exit graph
Dynamic Routing [11]	Multi-scale paths	Per image	Compute penalty	Learned gates	No measured accelerator constraint; dynamic paths
AutoSegEdge [15]	Searched architecture	Design	FLOPs + device latency	None	Search constraint; static architecture
PACE-Seg	Four elastic capacities	Per image	Measured complete route	Candidate-specific error	Hard measured budget; static compiled engines

The gap is per-image selection with candidate-specific error estimates and a budget defined by the measured complete accelerator route. PACE-Seg addresses this intersection with static compiled candidates; its contribution is integrative rather than a claim that its individual components are new.

3. Method

3.1. System overview

Shared training produces four elastic capacities, each exported as an independently compilable static graph. At inference, the tiny candidate supplies a segmentation and one entropy feature. Candidate-specific calibrators estimate error, measured route costs define feasibility, and the policy selects the cheapest feasible candidate predicted to meet the risk target or the lowest-risk feasible fallback. Figures 2–4 show the candidate set, visual path, and one complete decision.

3.2. Compiler-safe elastic candidate family

Tiny, small, medium, and large jointly vary width multiplier (0.25, 0.50, 0.75, 1.00), repeated-block depth (2, 3, 4, 6), and input resolution (384×192 , 512×256 , 768×384 , 1024×512). They share convolutional weights through active-channel slicing and retain level-specific batch-normalization state. The restricted operator set uses standard compiler-supported operations and fixed-factor resizing. Each level is exported as a separate static graph, so runtime routing selects an engine before execution and introduces no data-dependent control flow inside a compiled graph. Compiler compatibility is deployment evidence, not an accuracy contribution.

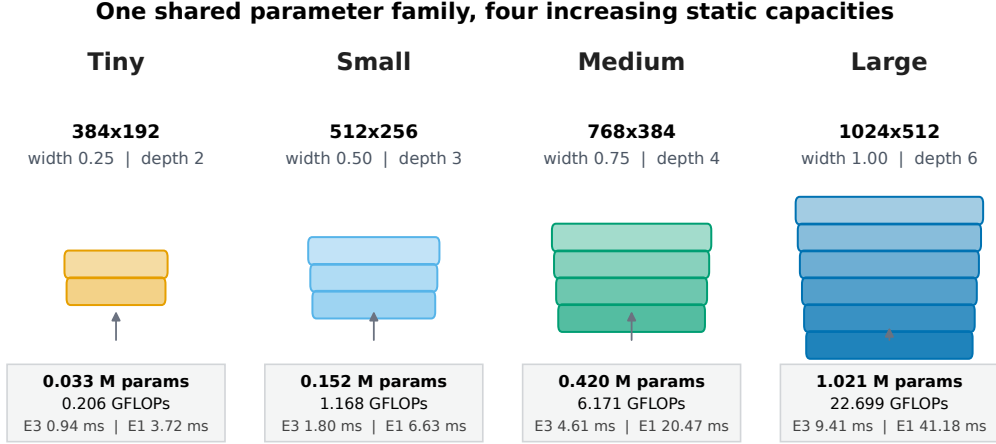


Figure 2: Elastic candidate family extracted from one shared model. The silhouettes increase jointly in input resolution, channel width, and repeated-block depth from tiny to large. The lower strip reports extracted parameters, GFLOPs, and candidate-only p95 latency on E3 TensorRT/CUDA and E1 Hailo-8; complete routing costs are measured separately.

All four capacities are extracted from one shared model rather than trained as unrelated networks. Capacity grows jointly in resolution, width, and depth. The measured latency increase differs from the GFLOPs increase and differs between backends.

3.3. Measured-cost candidate set

For hardware h , candidate l has measured route cost $t_{h,l}$. Candidate-only measurements characterize hardware scaling; runtime measurements include the tiny probe, entropy and policy computation, engine activation or dispatch, and any additional candidate inference. The tiny prediction is reused when tiny is selected. Cost enters after training, so a new target requires a four-entry measured table rather than segmentation retraining.

3.4. Candidate-specific visual-risk calibration

Let the tiny candidate produce logits $z(x)$. The raw probe score is mean pixel entropy,

$$s(x) = \frac{1}{|\Omega|} \sum_{u \in \Omega} - \sum_{c=1}^C p_{u,c} \log \max(p_{u,c}, 10^{-12}), \quad (1)$$

where p is the softmax of z and $C = 19$. The score is image-level and requires no separate probe network.

For each candidate l , a monotonic calibrator g_l maps the same $s(x)$ to observed per-image pixel error. Fitting and deployment compute entropy over all output pixels. Ground truth supplies only fit-half error targets over non-ignore labels. Ten equal-count probe-score bins provide mean candidate errors, followed by a cumulative-maximum pass. Thus

$$\hat{r}_l(x) = g_l(s(x)) \quad (2)$$

is candidate-specific calibration from shared sensing. The primary analysis configures one calibrator set per condition; a second variant pools all fit halves and requires no condition choice. Both use the same feature definition and held-out protocol.

3.5. Hardware-cost-conditioned policy

For a target device, candidate l has measured route cost t_l , risk target τ , and budget B . The proposed policy D forms the feasible set

$$\mathcal{S}_B = \{l : t_l \leq B\}. \quad (3)$$

If at least one feasible candidate satisfies $\hat{r}_l(x) \leq \tau$, D selects the cheapest such candidate. Otherwise it selects the feasible candidate with minimum predicted error, breaking ties by lower cost. If no candidate is within budget,

it selects the globally cheapest candidate. This fallback can still exceed a budget smaller than the cheapest available route; the evaluated budget grid uses the four measured route costs, so this case does not occur in the reported cells.

Policy A uses one probe calibrator and escalates by latency rank at integer multiples of τ . B maps the same risk to a latency target, and C uses candidate-specific risks without a hard budget. These progressive ablations are not a factorial design because only D adds explicit feasibility. We also evaluate pooled D, static candidates, raw-entropy routing, and a ground-truth oracle used only as a non-deployable upper bound.

Table 3: Routing-policy roles and information access. “Per condition” means that the deployment domain is configured before inference; pooled D removes that requirement. The oracle uses held-out ground truth and is not deployable.

Policy	Candidate risk	Hard budget	Calibrator	Deployable	Purpose
Static small/large	No	No	None	Yes	Fixed-capacity references
Raw entropy	No	No	None	Yes	Uncalibrated adaptive baseline
Policy A: rank escalation	No	No	Per condition	Yes	Rank-based comparator
Policy D: configured	Yes	Yes	Per condition	Yes	Primary method
Policy D: pooled	Yes	Yes	Pooled	Yes	Condition-agnostic check
Oracle	GT error	Yes	None	No	Upper bound only

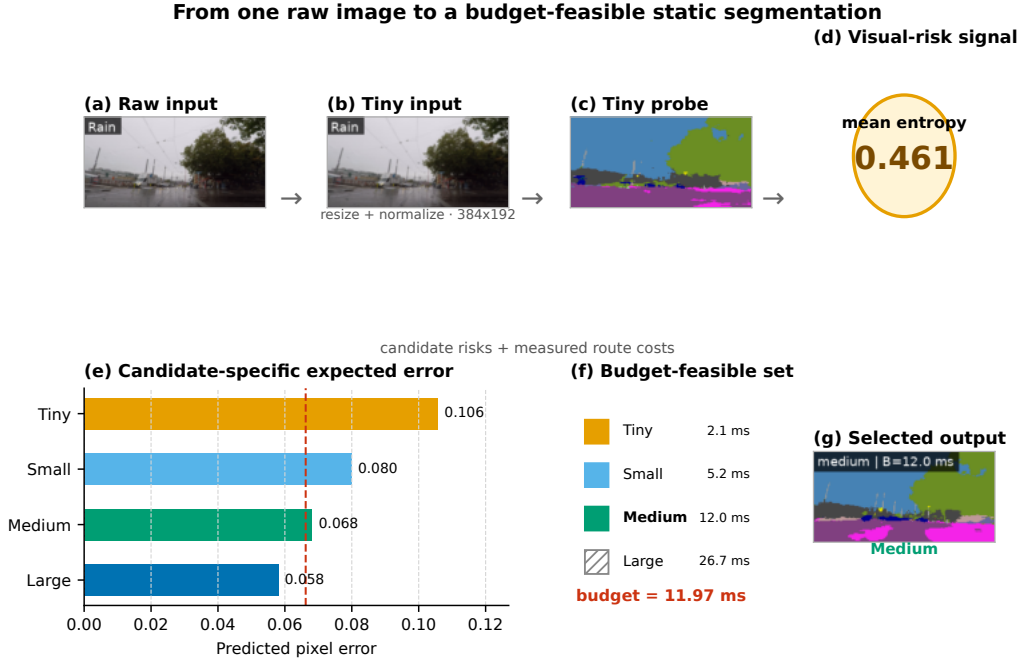


Figure 3: From one raw image to a routed segmentation. The image is prepared for the tiny probe, whose segmentation produces the shared entropy signal. Candidate-specific calibrators convert that signal into four expected errors; measured complete-route costs and the latency budget define the feasible set, from which policy D selects the displayed static-candidate output.

The probe is reused as the tiny output when tiny is selected. Risk estimation and hardware feasibility remain separate until the final decision. Figure 4 instantiates the same sequence with audited values.

From a raw adverse-condition image to one budget-feasible static output

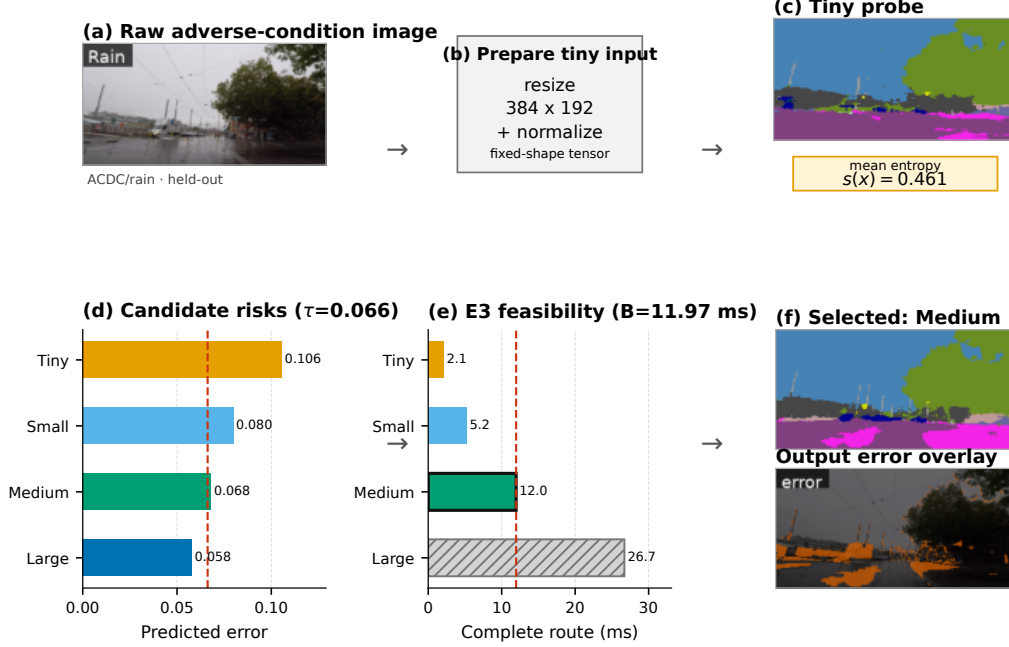


Figure 4: One complete policy-D decision on an audited rain image. Tiny-probe entropy maps to candidate-specific expected errors, while measured E3 complete route costs are compared with $B = 11.97$ ms. Large is infeasible and no feasible candidate meets $\tau = 0.066$, so the minimum-risk feasible fallback selects medium. The final panel overlays output-ground-truth disagreement in orange; this case illustrates the policy and is not an additional evaluation trial.

Risk alone favors large, which exceeds the budget. Cost alone cannot rank expected segmentation error among feasible candidates. Calibration orders feasible actions; the measured budget removes unavailable ones.

3.6. Implementation details and secondary QAT study

Elastic training uses the sandwich rule: each step evaluates the smallest and largest levels and one randomly selected intermediate level. The largest level receives supervised segmentation loss; smaller levels receive supervised and in-place distillation terms; and a boundary auxiliary term emphasizes

semantic transitions. Training uses 100,000 optimization steps, batch size 8, AdamW with weight decay 10^{-4} , initial learning rate 3×10^{-4} , cosine decay, 500 warm-up steps, and gradient clipping at 5.0. The data stream combines Cityscapes and ACDC with random scale/crop/pad, horizontal flip, and color jitter.

QAT replaces each convolution with per-tensor symmetric 8-bit fake quantization of its input activation and active weight tensor, using PyTorch’s straight-through estimator. Dynamic mode derives the activation scale from every call. Calibrated mode freezes one activation range per layer after a 200-image calibration pass spanning clean and adverse conditions. The maximum observer retains the largest activation observed over all calls. The EMA-percentile observer computes the 99.9th percentile of absolute activation in each call and combines successive values with momentum 0.9.

For a slimmable convolution, one calibration buffer belongs to the shared layer and aggregates calls from all elasticity levels; the implementation does not maintain a separate frozen activation scale for each level. Each QAT run fine-tunes 10,000 steps from an FP32 checkpoint at learning rate 3×10^{-5} . These are simulated fake-quant results. They neither execute integer kernels nor establish compiled TensorRT or Hailo INT8 accuracy.

4. Experimental protocol

4.1. Datasets and splits

Cityscapes supplies 2,975 training and 500 validation images [22]. ACDC supplies 1,600 training and 406 validation images distributed across fog, night, rain, and snow [21]. Both use the 19 Cityscapes train classes. Each validation split is partitioned by alternating index: even-index images form the calibration and operating-point fit half, and odd-index images form the held-out

evaluation half. The study does not use an external test server.

Table 4: Dataset composition and alternating-index router split. Fit and held-out counts partition each validation subset exactly.

Dataset/condition	Train	Validation	Fit	Held-out
Cityscapes clean	2,975	500	250	250
ACDC fog	400	100	50	50
ACDC night	400	106	53	53
ACDC rain	400	100	50	50
ACDC snow	400	100	50	50

Figure 1 introduced these five domains in the Introduction. Table 4 supplies the exact counts and makes explicit that calibration and operating-point selection use only the fit half, whereas the reported router outcomes use the held-out half.

4.2. Training runs and baselines

The near-parity comparison uses three reported supernet runs and three augmented Fast-SCNN runs. The router analysis uses the three separately trained checkpoints summarized in Table 5. The deployment-matched artifacts record each checkpoint SHA-256 and its embedded configuration hash. Initialization-parent identifiers and immutable full training manifests remain unavailable, so we call them separate training runs rather than a controlled identical-configuration seed study.

Table 5: Training-checkpoint provenance used by the router analysis. Hashes are abbreviated; full SHA-256 values and remaining provenance boundaries are retained in the evidence manifest.

Run	Training record	Step	Checkpoint SHA	Config hash
A	Augmentation-labelled	100,000	a2444156	6d1274f40e43
B	Earlier unlabelled	100,000	00da283c	837cf981427a
C	Augmentation-labelled	100,000	275dfcbf	ac52eabac3de

Fixed-architecture comparisons cover Fast-SCNN, BiSeNetV2, SegFormer-B0, DDRNet-23-slim, and MobileNetV3/DeepLabV3. Fast-SCNN is the parameter-matched RQ2 comparator for the large elastic level. Routing references comprise static candidates, raw entropy, rank-based policy A, latency-spacing policy B, candidate-specific policy C without a budget, condition-specific and pooled variants of constrained policy D, and an oracle.

4.3. Hardware and compiler versions

Table 6 identifies the four latency targets and their compiler stacks. TensorRT measurements use GPU FP16. Hailo candidates are compiled from ONNX through Hailo’s Dataflow Compiler to HEF. Compiler smoke tests establish that all four static candidates execute on the primary TensorRT GPU and Hailo paths; DLA is excluded from headline analysis because decoder execution falls back to the GPU.

Table 6: Hardware and compiler stacks used for candidate-latency measurement. Versions are reported because engine behavior is toolchain-specific.

ID	Hardware	Runtime and system versions
E1	Raspberry Pi 5 + Hailo-8 M.2	HailoRT 4.23.0; HEFs produced with Dataflow Compiler 3.34.0 and Model Zoo 2.19.0
E2	Jetson Xavier NX	L4T R35.4.1; CUDA 11.4.19; cuDNN 8.6.0.166; TensorRT 8.5.2.2
E3	Jetson AGX Xavier	L4T R35.6.4; CUDA 11.4.19; TensorRT 8.5.2.2
E5	Jetson Orin Nano Super	L4T R36.5.0; CUDA 12.6.68; cuDNN 9.3.0; TensorRT 10.3.0.30

4.4. Latency protocol

Candidate-only latency is measured on four devices: E1, a Raspberry Pi 5 host with a Hailo-8 M.2 accelerator; E2, a Jetson Xavier NX; E3, a Jetson AGX Xavier; and E5, a Jetson Orin Nano. NVIDIA devices use TensorRT FP16. Measurements use batch size one, warm-up, three independent benchmark runs for the candidate lookup table, and bootstrap intervals. Energy is excluded because no external calibrated power meter was available.

Complete warm route costs are measured on E1 and E3. On E3, all four TensorRT contexts are resident; route timing includes tiny inference, device-resident entropy, policy evaluation, dispatch, and additional candidate inference. On E1, route timing includes mandatory network-group activation and deactivation, tiny inference, host entropy, policy evaluation, and additional candidate inference. Each route class uses 50 warm-up iterations and at least 500 timed iterations. Tiny output is reused when tiny is selected.

Backend-specific cold-start measurements are excluded because the two run-times impose different reload semantics.

The route harnesses use compiled graphs with the correct post-rescale architecture but do not execute trained checkpoint weights for the accuracy evaluation. The analysis combines measured route latency with cached trained-model predictions in an offline, per-image policy replay. It is not an integrated on-device accuracy run.

4.5. Metrics, seed policy, and unit of analysis

Segmentation quality is dataset-level mIoU. Router calibration targets per-image pixel error, while routed quality is recomputed by accumulating confusion matrices from the selected candidates and calculating mIoU once over the routed held-out set. Headline accuracy comparisons require three training runs. The router evidence uses Runs A–C, but incomplete configuration provenance prevents treating them as a perfectly controlled identical-configuration seed study.

The router reporting unit is an operating cell defined by checkpoint, backend cost table, dataset split, and budget. Cells reuse images, predictions, thresholds, and candidate sets. They are correlated analysis points, not independent statistical samples. We therefore report descriptive counts and macro averages without cell-level p -values or confidence intervals.

A fair A-versus-D cell is one in which A has zero held-out per-image budget violations at its fit-selected operating point. An operating cell is violating when at least one held-out image exceeds its budget. Fair-cell quality counts are always paired with violation counts over the full grid. D’s zero violating-cell count is partly structural because D removes candidates above budget.

4.6. *Fit/held-out separation and leakage safeguards*

Every candidate is evaluated once per image and its prediction is cached. The probe feature is mean softmax entropy over all output pixels in both fitting and deployment. Within each dataset split, condition-specific calibrators use only fit-half probe scores and candidate errors; the pooled variant combines only those fit halves across the five splits. Five risk targets are the 10th, 25th, 50th, 75th, and 90th percentiles of fit-half observed error, macro-aggregated across the five splits. One grid is generated per checkpoint and reused across E1 and E3 cost tables.

For each budget and policy, the representative risk target is selected from fit-half statistics: among points whose fit-half mean cost satisfies the budget, the procedure chooses the highest fit-half mIoU and breaks ties by lower cost. Held-out outcomes are read only after this choice. The oracle uses held-out ground truth only as a non-deployable upper bound.

Held-out labels do not fit calibrators, define thresholds, or select operating points. They are used only to reconstruct final routed mIoU and to report per-image budget violations after selection. The all-pixel probe feature requires no ground-truth mask at deployment. The historical masked-fit artifacts are retained for audit but are not used for the RQ3 headline.

5. Results

5.1. *RQ1: Why measured hardware cost is necessary*

The four levels require 0.2060, 1.1678, 6.1712, and 22.6992 GFLOPs. Their large-to-tiny ratio is 110.2, whereas measured mean end-to-end candidate latency grows by only $10.3\text{--}19.5\times$ across the four accelerators (Fig. 5). A 10 ms budget selects small on E1/E2, large on E3, and medium on E5. Measured cost is therefore necessary to recover hardware feasibility in this

candidate set. The result concerns a proportional FLOPs proxy; it does not establish that learned latency predictors or FLOPs-aware search fail in general.

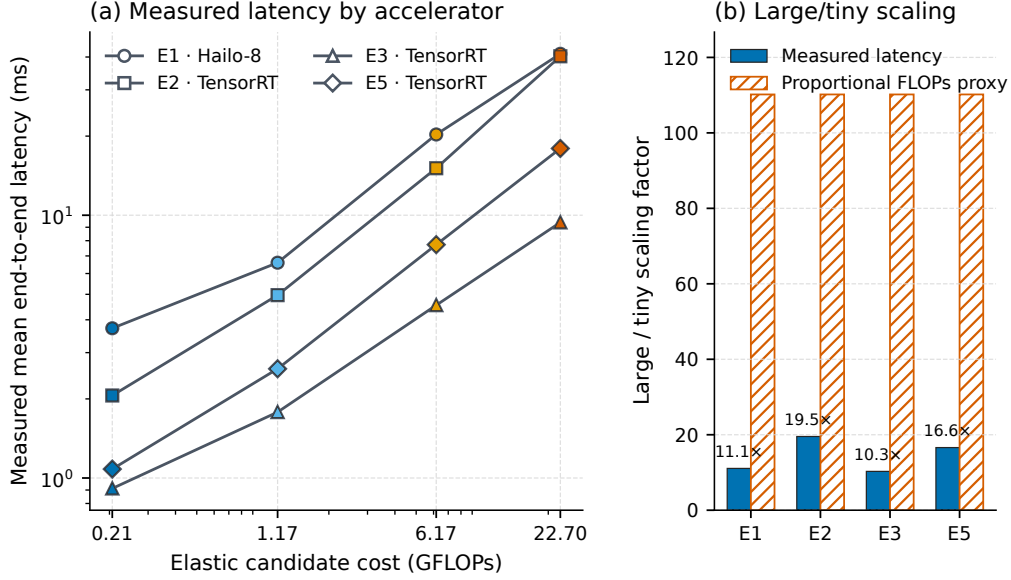


Figure 5: Why FLOPs is insufficient as a deployment-cost proxy. Marker shape identifies hardware and color identifies capacity. The $110.2\times$ large-to-tiny compute increase becomes only $10.3\text{--}19.5\times$ measured latency, so FLOPs preserves order here but not the hardware-specific scale needed to enforce a latency budget.

The budget sweep confirms that the mismatch affects selection rather than only scale. Across 640 proxy evaluations, same-device proportional fits mis-select 37.5–65% of budgets, and the worst cross-device transfers reach 85–95% mis-selection or violation. Measured cost is therefore required for this candidate set. The sweep does not compare against learned latency predictors.

5.2. RQ2: Whether shared elastic training preserves quality

The elastic model preserves matched-capacity segmentation quality across clean and adverse conditions. Across the three reported runs per model, the large level differs from a similarly sized independently trained Fast-SCNN by +1.10, +0.21, -0.65, -0.23, and +0.07 mIoU points on Cityscapes, fog, night, rain, and snow, respectively (Fig. 6). Every observed mean difference remains within the prespecified ± 1.5 -point practical near-parity band, and the run envelopes cross zero on four of five splits. This supports practical near-parity rather than superiority. The comparison does not measure total training, packaging, or maintenance savings.

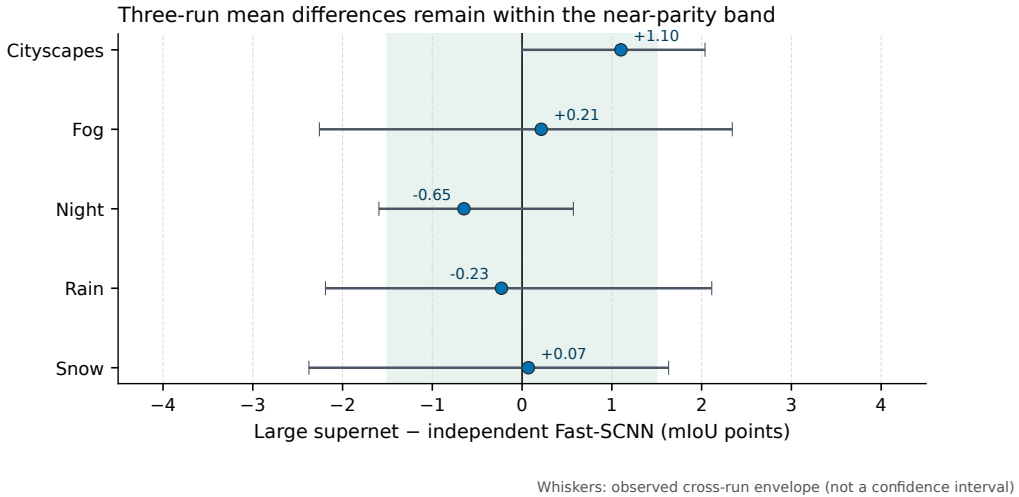


Figure 6: Three-run matched-capacity comparison. Points show mean mIoU differences between the large elastic candidate and independently trained Fast-SCNN; whiskers are the full observed cross-run envelopes, not confidence intervals or paired-seed estimates. All means lie within the prespecified ± 1.5 -point practical near-parity band, supporting near parity rather than superiority or statistical equivalence.

5.3. RQ3: Candidate-specific routing under measured budgets

The routing study tests whether candidate-specific expected-error estimates improve budget-constrained selection over a single-score rank policy. An earlier candidate-only ablation supports the progression from A to D, but it omits complete route overhead and predates deployment-matched calibration. We therefore use it only as mechanism-level directional evidence and base the RQ3 conclusion on the corrected complete-route replay below.

5.3.1. Measured route costs

Complete route costs materially exceed candidate-only inference costs, and the dominant source depends on the accelerator. Table 7 reports the warm-route medians used in the overhead-aware replay; every value includes the tiny probe, risk evaluation, policy decision, switching or activation, and any additional candidate inference.

Table 7: Median warm end-to-end route cost used in the overhead-aware replay.

Route	E3 TensorRT/CUDA	E1 Hailo-8
tiny $\rightarrow$ tiny	2.07 ms	34.95 ms
tiny $\rightarrow$ small	5.23 ms	46.13 ms
tiny $\rightarrow$ medium	11.97 ms	63.36 ms
tiny $\rightarrow$ large	26.70 ms	92.68 ms

On E3, complete routes range from 2.07 to 26.70 ms; on E1 they range from 34.95 to 92.68 ms. Host-side uncertainty evaluation is the main avoidable cost on E3, whereas network-group activation dominates E1. A production-path audit on 40 TensorRT outputs measures a maximum risk-score deviation of 7.15×10^{-7} nats from the PyTorch reference, below the prespecified 10^{-4} tolerance. The measurements show that candidate-only

lookup tables omit substantial, backend-specific costs. They cover two warm execution paths and do not establish equivalent cold-start semantics across the backends.

Figure 7 applies the same audited risk vector and illustrative 60 ms budget to both measured backends.

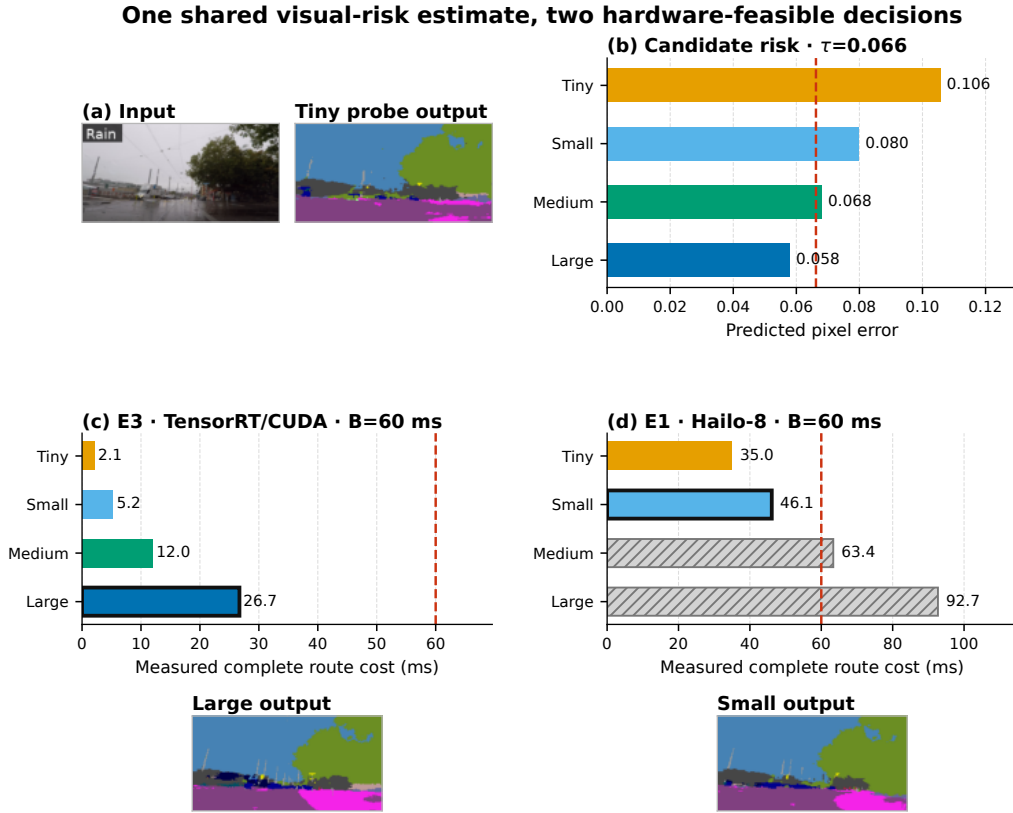


Figure 7: Same image and risk vector, different measured hardware costs. Under the illustrative 60 ms budget, all E3 routes are feasible and policy D selects large; E1 excludes medium and large and selects small. Only the cost table changes, and the example is a policy illustration rather than an additional evaluation cell.

All four routes fit the budget on E3, and large is the cheapest candidate meeting the risk target. On E1, only tiny and small are feasible, so the fallback selects small as the lower-risk option. The changed action is caused

by measured hardware cost, not a changed image or risk estimate.

5.3.2. Deployment-matched routing result

The overhead-aware replay recomputes A and D decisions with measured warm route costs, constructs each backend’s budget grid from its four route costs, and selects risk targets using fit-half data. One canonical risk grid per training run is shared across E1 and E3. The probe feature is identical at fitting and deployment. We report both configured condition-specific D and pooled D, which fits one candidate-calibrator set from all five fit halves.

Table 8: Deployment-matched A-versus-D comparison with measured complete route costs. W/T/L uses fair cells; violations use the full grid. Pooled D uses one calibrator set across conditions.

Policy	Backend	Fair	W/T/L	Win/tie	Mean Δ	Policy viol.	A viol.
Configured D	E3	38/60	29/7/2	94.7%	+0.0287	0/60	22/60
Configured D	E1	36/60	27/7/2	94.4%	+0.0279	0/60	24/60
Configured D	Overall	74/120	56/14/4	94.6%	+0.0284	0/120	46/120
Pooled D	Overall	74/120	58/14/2	97.3%	+0.0296	0/120	46/120

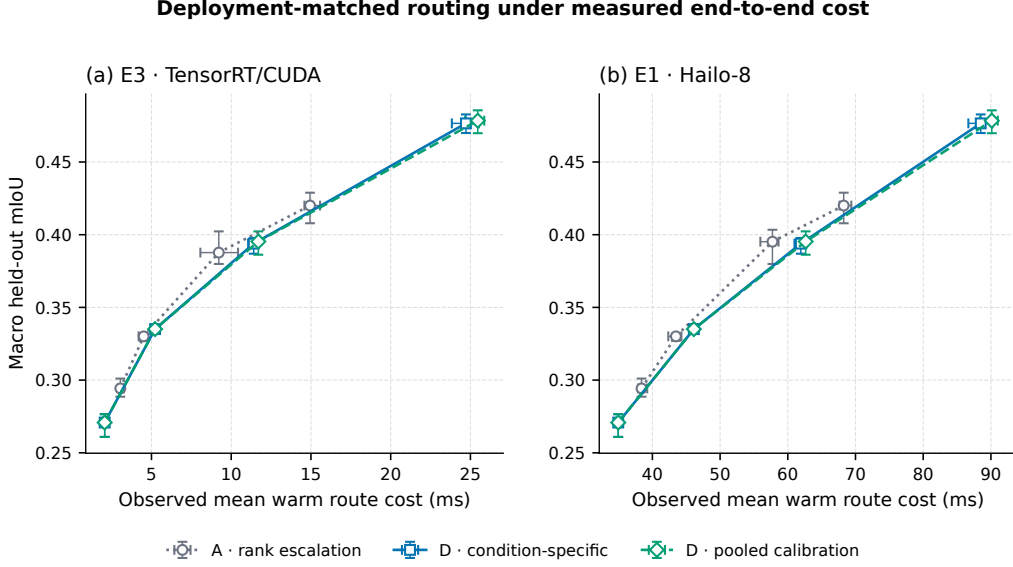


Figure 8: Quality versus measured complete warm-route cost after deployment-matched calibration. Points macro-average five held-out splits within each run and then Runs A–C; whiskers show the run-level range, not confidence intervals. Configured and pooled policy D reach higher-quality operating points than rank policy A on both measured backends.

Under this protocol, configured D outperforms or matches A in 70 of 74 fair cells (56 wins, 14 ties, and 4 losses), with a macro D-minus-A gap of $+0.0284$ mIoU. Across the full 120-cell grid, D has no budget-violating operating point and A has 46. Pooled D records 58 wins, 14 ties, and 2 losses on the same fair subset, with a $+0.0296$ gap. The similar curves in Fig. 8 show that candidate-specific hard-budget selection, rather than per-condition calibrator choice, drives the observed gain.

The 74 fair cells are the subset in which A satisfies its own budget; they are correlated operating points rather than independent trials. D’s zero-violation count follows partly from enforcing feasibility by construction, and E1 and E3 share segmentation predictions. The cross-backend comparison therefore tests the effect of different measured cost tables, not independent

accuracy replication. Absolute hardware costs determine feasibility and latency, but identical candidate ordering can still produce identical decisions.

5.4. *Adverse-condition behavior*

Adverse conditions affect both segmentation quality and the informativeness of the shared probe. Across Runs A–C, the large elastic level averages 0.5311 mIoU on the held-out Cityscapes half but 0.3559 on ACDC/night. Probe-signal AURC averages 0.1411 across the fifteen run-split evaluations and is worst at night, where the three-run mean is 0.2416. Night imagery therefore weakens both prediction quality and uncertainty ranking. These are descriptive values on the evaluated validation halves.

Figure 9 provides a qualitative Run A view of the same five deterministically selected held-out images. Increasing capacity changes object boundaries and small-region predictions, but errors remain visible under every condition and are broadest in the night example. At the fixed E3 complete-route budget of 11.97 ms, configured D selects medium for all five examples because no in-budget candidate meets the chosen risk target and medium has the lowest predicted error among the feasible candidates. The routed column therefore verifies the policy output at this operating point; it does not demonstrate route diversity or cross-run replication.

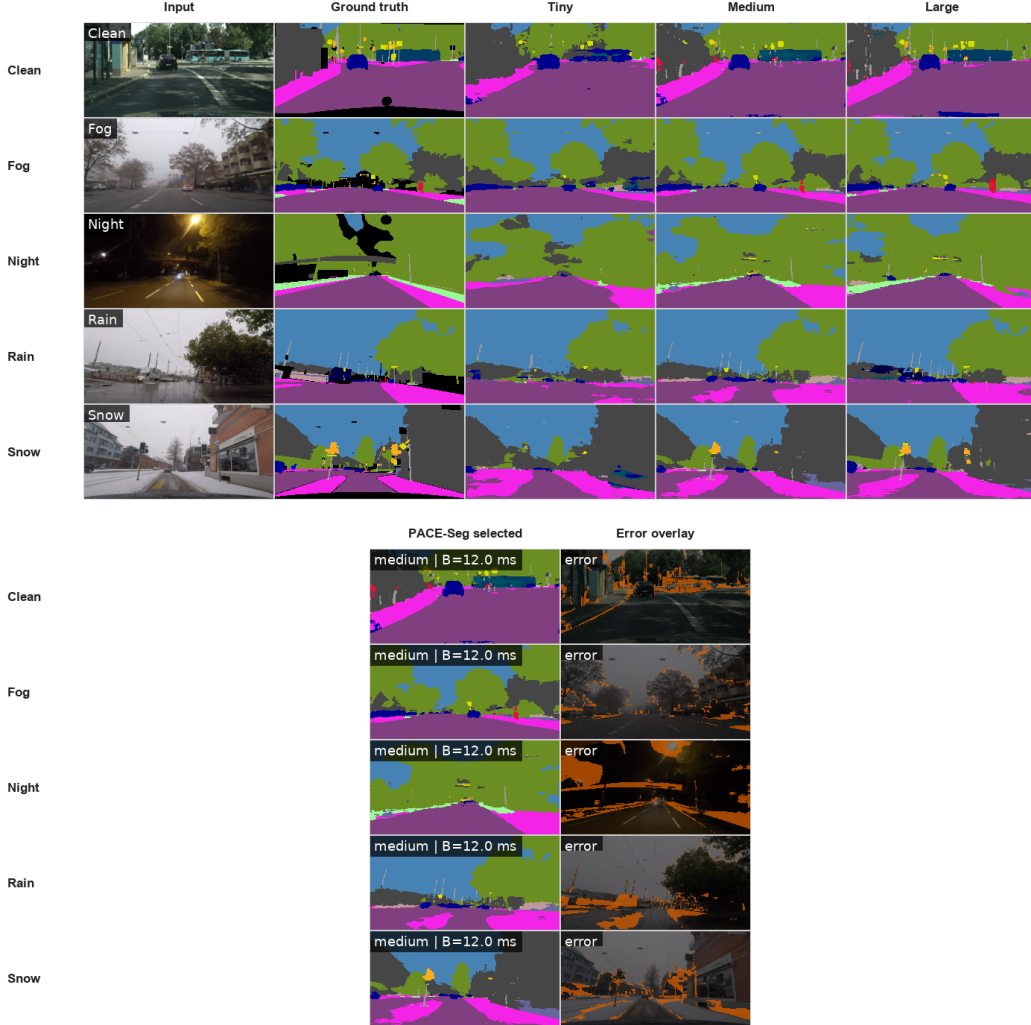


Figure 9: Qualitative capacity comparison on the five held-out examples from Fig. 1. The upper block compares input, ground truth, tiny, medium, and large predictions; the lower block enlarges the configured policy-D output and its error overlay, where orange marks disagreement with ground truth. At $B = 11.97$ ms on E3, D selects medium for all five images. Image selection was fixed by median large-candidate pixel error before visual inspection.

Configured D’s four fair-cell losses arise from two Run C operating points—fog and rain at the medium budget—replayed on both backends. Pooled D’s two small losses are the same Run B snow operating point

replayed on E1 and E3. The duplication follows shared predictions and should not be interpreted as independent failures or a failure probability.

5.5. Secondary deployment and fake-quant evidence

All four static candidate graphs compile and execute through TensorRT and Hailo-8 deployment paths. This establishes compiler compatibility for the constrained operator set, but the study has no trained unconstrained control and therefore does not attribute an accuracy benefit to compiler safety.

Observer choice controls the worst adverse-condition degradation in the fake-quant study. Dynamic fake quantization of the shared large level loses 1.66, 2.54, and 1.39 mIoU points in the worst condition of the three runs, whereas EMA-percentile calibration produces the bounds in Table 9. A Run A 2×2 comparison indicates a larger change from the observer than from subnet export. Because the identity of the worst condition can differ between cells and mean effects are not uniformly positive, the comparison does not establish additive causal effects.

Table 9: Absolute worst-condition mIoU degradation under shared-supernet EMA-percentile fake-quant QAT.

Training run	tiny	small	medium	large
Run A	0.32	0.97	0.51	0.97
Run B	0.69	0.33	0.49	1.38
Run C	0.36	0.33	0.56	0.86

Every evaluated run and level remains within 1.5 points, and the worst large-level run improves from 2.54 to 1.38 points. EMA-percentile calibration therefore stabilizes worst-condition fake-quant degradation in this set-

ting. Mean quality is not consistently improved, activation outlier suppression remains an untested explanation, and no compiled INT8 accuracy claim follows.

6. Discussion

The experiments separate candidate construction, hardware feasibility, and image-dependent selection. Shared elastic training supplies four static candidates without a large quality penalty at matched capacity. Direct measurement then defines which candidates fit a deployment budget. Candidate-specific calibration finally estimates whether additional capacity is warranted for an image. Treating these as distinct decisions makes clear which evidence supports each part of the system.

FLOPs fails here primarily as a cost scale rather than an ordering statistic. The 110.2-fold compute range contracts to a 10.3–19.5-fold latency range because operators, memory behavior, compiler mappings, and accelerator execution differ. A ranking may therefore transfer when candidate order is preserved, while budget feasibility still changes with absolute measured cost. This explains why some routing decisions coincide across devices without making the cost table dispensable.

Candidate-specific calibration addresses a limitation of a single probe score: equal probe entropy need not imply equal error for every candidate. The deployment-matched results are consistent with this premise, but calibration cannot recover visual information absent from the probe. ACDC/night has the weakest uncertainty ordering even though the residual paired losses occur in other conditions. Pooled calibration matches or slightly exceeds configured condition-specific calibration, so condition-specific fitting is not necessary for the reported result; generalization to conditions absent from

the calibration pool remains untested.

Complete route measurement changes the deployment conclusion. Candidate-only inference time omits uncertainty computation and accelerator-specific switching or activation. These omissions are large on both evaluated backends even though their causes differ. Hardware-aware runtime selection should therefore account for the complete route path, not only isolated candidate latency.

The quantization study reinforces the need to separate measurement from mechanism. Percentile/EMA calibration reduces worst-condition fake-quant degradation across the tested runs and levels, but the data do not directly measure activation outliers or compiled integer behavior. QAT consequently remains secondary deployment evidence rather than a primary contribution.

7. Limitations and threats to validity

First, calibration is evaluated only on Cityscapes and the four ACDC conditions. The configured variant assumes that the condition is known, while the pooled variant removes that runtime choice but still uses fit-half samples from every evaluated condition. Neither result establishes calibration on a previously unseen adverse condition.

Second, the evaluation uses alternating halves of validation splits rather than an external test set. Multiple budgets and thresholds reuse the same images and predictions. Operating cells are correlated, and the 120-cell denominator is not a statistical sample size. Runs A–C form the training-replication axis. Checkpoint and embedded configuration hashes are recorded, but initialization-parent provenance is incomplete.

Third, fair-cell conditioning is descriptive. It excludes A’s budget-violating comparisons so that quality is compared only when A honors

its contract, but the denominator depends on baseline behavior. Full-grid violation counts must accompany the fair-cell result. D’s zero violations are partly guaranteed by construction and are not empirical reliability.

Fourth, E1 and E3 share segmentation predictions. Separately measured route-cost tables are combined with the same per-image prediction caches. The compiled timing engines have the correct architecture but are not the trained engines that produced accuracy. The study validates cost accounting and overhead-aware replay, not bit-exact compiled-model accuracy or a continuously running application.

Fifth, breakpoint budgets can reduce hardware conditioning to latency rank. Absolute costs change feasibility and mean-latency operating-point selection, but identical orderings can produce identical route choices. Hardware-cost-conditioned describes the policy input; it does not imply device-specific decisions in every case.

Sixth, QAT uses fake quantization in PyTorch. No compiled INT8 accuracy, integer-kernel speedup, or backend-specific quantization behavior is established. The observed EMA-percentile result is consistent with reduced outlier sensitivity but does not measure that mechanism directly.

Finally, the study uses four candidates, two complete route-cost backends, and no external power meter. It makes no energy claim. Temporal routing, unknown-condition calibration, cold-frontier behavior, uncertainty-aware segmentation metrics, and systematic class- or boundary-level qualitative analysis remain outside scope. The compiler-safe operator set was validated without a trained unconstrained control, so portability is an engineering property rather than an independently isolated accuracy contribution.

8. Conclusion

PACE-Seg studies elastic semantic segmentation when both visual difficulty and accelerator cost vary. Measured latency is necessary to define feasible candidates, and shared elastic training preserves matched-capacity quality across the evaluated clean and adverse conditions. With deployment-matched calibration and complete warm-route costs, candidate-specific hard-budget routing improves the descriptive quality–cost trade-off over rank escalation on two structurally different accelerators. Pooled calibration retains the result without condition selection, while correlated cells, shared predictions, offline replay, and the absence of an external test set bound the claim.

Data and code availability

The repository records the implementation, experiment reports, and canonical result artifacts. The accompanying evidence manifest maps Runs A–C to checkpoint and configuration hashes and traces each figure and numerical claim to its source artifact. The deployment-matched replay directory is the canonical RQ3 evidence; historical masked-fit replays remain available only for audit. Dataset access remains subject to the Cityscapes and ACDC licenses.

Appendix A. Qualitative error gallery

Figure S1 combines the five deterministic median-error split representatives with the predetermined highest-error Run A ACDC case. It illustrates error structure and does not estimate a failure rate.

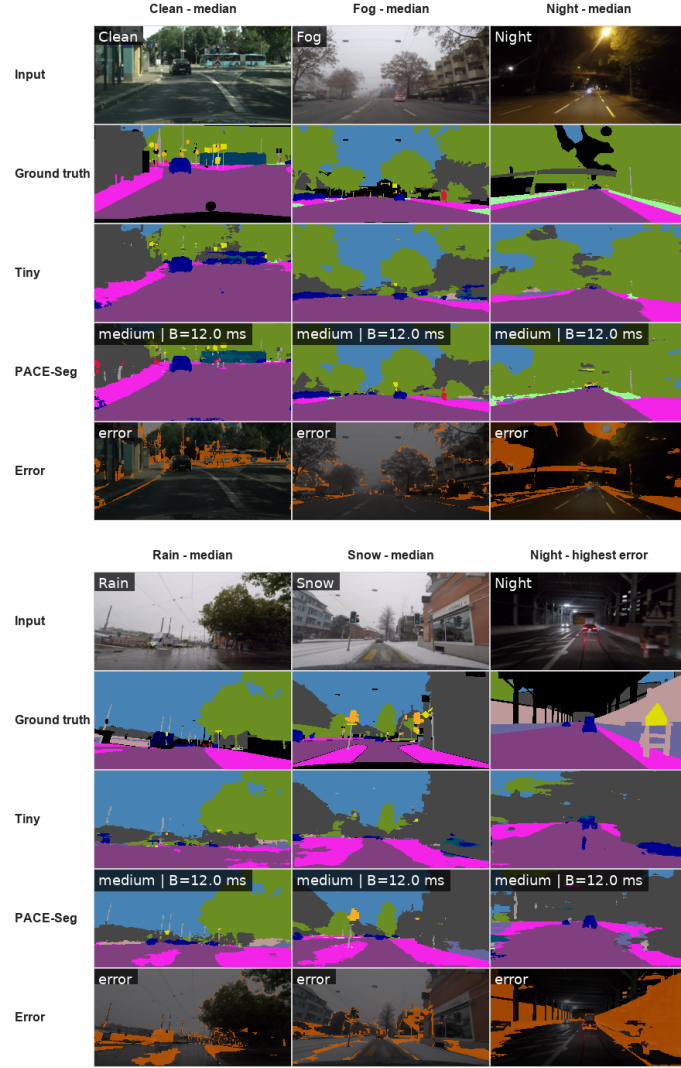


Figure S1: Expanded qualitative error gallery. The six audited examples comprise the five deterministic median-error split representatives and the predetermined highest-error Run A ACDC night case. Each example compares input, ground truth, tiny, configured policy-D output, and output error; orange marks disagreement with ground truth. The gallery audits error structure and does not estimate a failure rate.

The median cases show recurring boundary and small-region errors without selecting examples by visual success. The additional night case is deliberately difficult under the predetermined highest-error rule. All routed outputs

use the configured policy at the displayed operating point.

Appendix B. Complete-route cost expansion

Figure S2 compares isolated candidate latency with the complete warm route used by the router replay. The gap is largest for the low-capacity Hailo-8 routes because activation and host-side work dominate their short isolated inference time. The bars compare two measured totals; they are not a component decomposition.

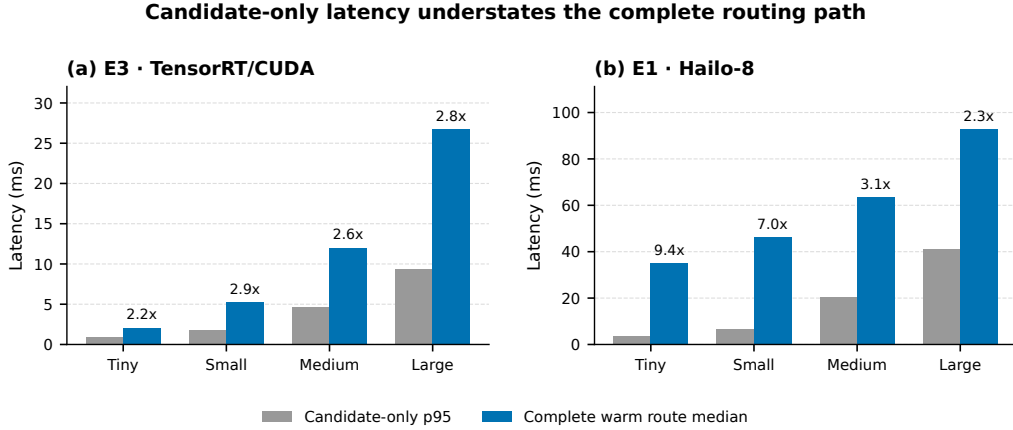


Figure S2: Candidate-only p95 latency versus measured complete warm-route median. The complete route includes the tiny probe, risk computation, policy evaluation, backend dispatch or activation, and any additional candidate inference. Ratios above the blue bars quantify the expansion relative to isolated candidate p95. Different summary statistics are named explicitly; the figure supports the scope of the routing-cost model rather than a component-level causal attribution.

Appendix C. Additional audited qualitative views

Figures S3 and S4 reorganize the same six audited examples used in Fig. S1. They do not introduce condition-specific easy/hard samples or additional evaluation observations.

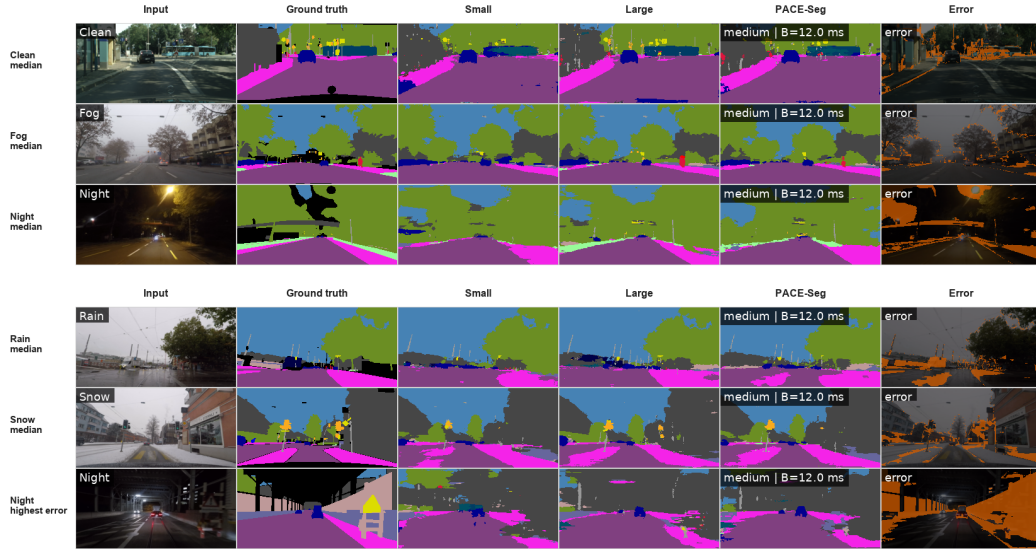


Figure S3: Additional full-frame comparison of the six audited examples. Columns show input, ground truth, small, large, configured policy-D output, and selected-output error. The panel exposes capacity-dependent changes while preserving the frozen image set; it is not an easy-versus-hard sampling study.

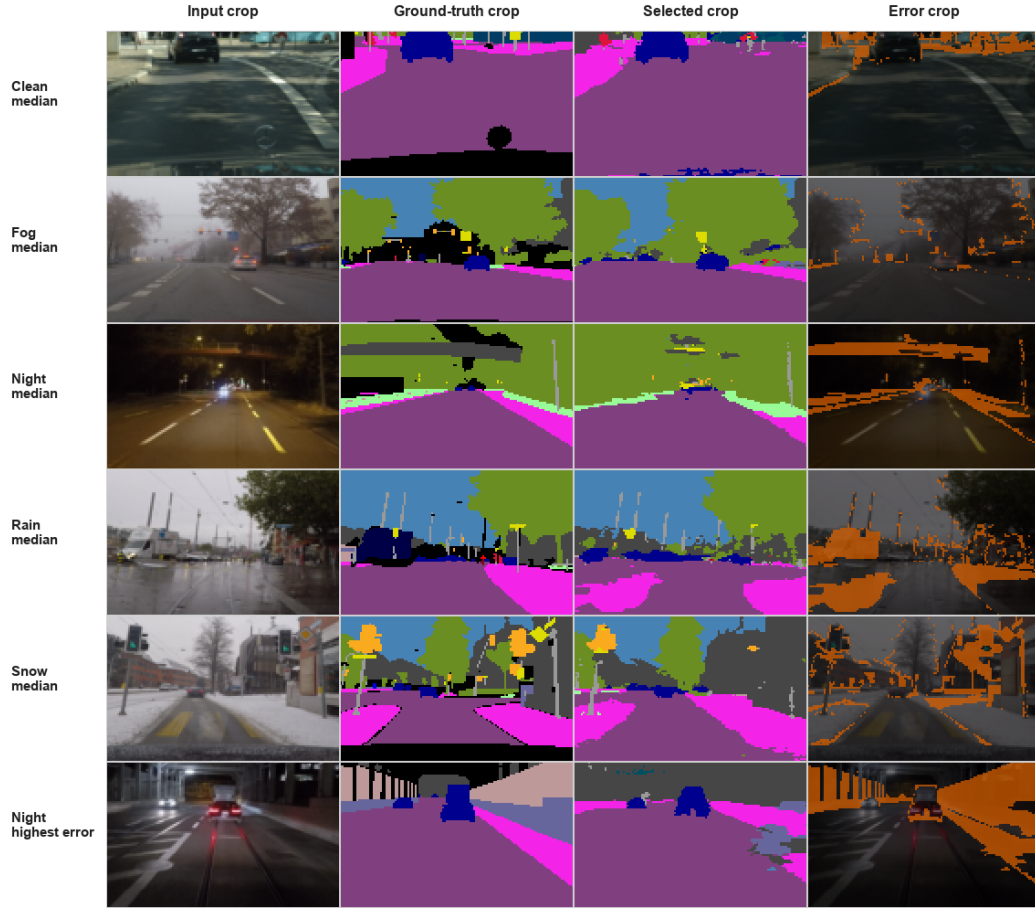


Figure S4: Local error crops from the same six audited examples. Every crop uses the identical normalized window $x = [0.25, 0.75]$ and $y = [0.36, 1.00]$, rather than image-specific manual framing. The fixed view enlarges boundaries, thin structures, vehicles, and pedestrians without changing predictions or selecting regions by visual success.

Appendix D. Condition-level routing summary

Table S1 disaggregates the canonical deployment-matched comparison by visual condition. Counts pool three training runs and two backend cost tables, so cells remain correlated and E1/E3 share segmentation predictions. Fair cells are those in which policy A has zero held-out budget violations.

Table S1: Condition-level configured and pooled D comparisons against policy A. W/T/L and mean Δ use fair cells; violation counts use all 24 cells per condition. Configured and pooled D each have zero violations, so one D column is shown. These are descriptive correlated operating cells, not independent samples.

Condition	Fair	Config. W/T/L	Config. Δ	Pooled W/T/L	Pooled Δ	A viol.	D viol.
Cityscapes	7/24	7/0/0	+0.0610	7/0/0	+0.0612	17/24	0/24
ACDC/Fog	20/24	14/4/2	+0.0373	16/4/0	+0.0392	4/24	0/24
ACDC/Night	11/24	11/0/0	+0.0125	11/0/0	+0.0125	13/24	0/24
ACDC/Rain	24/24	12/10/2	+0.0248	14/10/0	+0.0271	0/24	0/24
ACDC/Snow	12/24	12/0/0	+0.0160	10/0/2	+0.0157	12/24	0/24

References

- [1] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, N. Sang, BiSeNet: Bilateral segmentation network for real-time semantic segmentation, in: European Conference on Computer Vision, 2018, pp. 325–341. https://openaccess.thecvf.com/content_ECCV_2018/html/Changqian_Yu_BiSeNet_Bilateral_Segmentation_ECCV_2018_paper.html.
- [2] M. Fan, S. Lai, J. Huang, X. Wei, Z. Chai, J. Luo, X. Wei, Rethinking BiSeNet for real-time semantic segmentation, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 9716–9725. https://openaccess.thecvf.com/content/CVPR2021/html/Fan_Rethinking_BiSeNet_for_Real-Time_Semantic_Segmentation_CVPR_2021_paper.html.
- [3] J. Xu, Z. Xiong, S.P. Bhattacharyya, PIDNet: A real-time semantic segmentation network inspired by PID controllers, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 19529–19539. <https://openaccess.thecvf.com/content/CVPR2023/html/>

Xu_PIDNet_A_Real-Time_Semantic_Segmentation_Network_Inspired_by_PID_Controller_CVPR_2023_paper.html.

- [4] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J.M. Alvarez, P. Luo, SegFormer: Simple and efficient design for semantic segmentation with transformers, *Advances in Neural Information Processing Systems* 34 (2021) 12077–12090.
- [5] S.-C. Zhang, Y. Li, Y.-H. Wu, Q. Hou, M.-M. Cheng, Revisiting efficient semantic segmentation: Learning offsets for better spatial and class feature alignment, in: *IEEE/CVF International Conference on Computer Vision*, 2025, pp. 22361–22371. https://openaccess.thecvf.com/content/ICCV2025/html/Zhang_Revisiting_Efficient_Semantic_Segmentation_Learning_Offsets_for_Better_Spatial_and_ICCV_2025_paper.html.
- [6] J. Yu, L. Yang, N. Xu, J. Yang, T. Huang, Slimmable neural networks, in: *ICLR Workshop*, 2019. <https://arxiv.org/abs/1812.08928>.
- [7] J. Yu, T. Huang, Universally slimmable networks and improved training techniques, in: *IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1803–1811. <https://doi.org/10.1109/ICCV.2019.00189>.
- [8] H. Cai, C. Gan, T. Wang, Z. Zhang, S. Han, Once-for-All: Train one network and specialize it for efficient deployment, in: *International Conference on Learning Representations*, 2020. <https://openreview.net/forum?id=HylxE1HKwS>.
- [9] D. Xue, F. Yang, P. Wang, L. Herranz, J. Sun, Y. Zhu, Y. Zhang, SlimSeg: Slimmable semantic segmentation with boundary supervision, in:

- Proceedings of the 30th ACM International Conference on Multimedia, 2022, pp. 6539–6548. <https://doi.org/10.1145/3503161.3548191>.
- [10] A. Kouris, S.I. Venieris, S. Laskaridis, N.D. Lane, Multi-exit semantic segmentation networks, in: European Conference on Computer Vision, 2022, pp. 330–349.
 - [11] Y. Li, L. Song, Y. Chen, Z. Li, X. Zhang, X. Wang, J. Sun, Learning dynamic routing for semantic segmentation, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 8553–8562.
 - [12] Z. Liu, Z. Xu, H.-J. Wang, T. Darrell, E. Shelhamer, Anytime dense prediction with confidence adaptivity, in: International Conference on Learning Representations, 2022. <https://openreview.net/forum?id=kNKFOX1euC>.
 - [13] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, Q.V. Le, MnasNet: Platform-aware neural architecture search for mobile, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2820–2828.
 - [14] W. Chen, X. Gong, X. Liu, Q. Zhang, Y. Li, Z. Wang, FasterSeg: Searching for faster real-time semantic segmentation, in: International Conference on Learning Representations, 2020. <https://openreview.net/forum?id=BJgqQ6NYvB>.
 - [15] Z. Dou, D.-Y. Ye, B. Wang, X. Wang, S. Sun, AutoSegEdge: Searching for the edge device real-time semantic segmentation based on multi-task learning, Image and Vision Computing 136 (2023) 104719. <https://doi.org/10.1016/j.imavis.2023.104719>.

- [16] Z. Dou, D.-Y. Ye, B. Wang, X. Wang, S. Sun, Multi-objective neural architecture search for efficient and fast semantic segmentation on edge, *IEEE Transactions on Intelligent Vehicles* 9 (1) (2024) 1346–1357. <https://doi.org/10.1109/TIV.2023.3332594>.
- [17] Y. Li, C. Yang, P. Zhao, G. Yuan, W. Niu, J. Guan, H. Tang, M. Qin, Q. Jin, B. Ren, X. Lin, Y. Wang, Towards real-time segmentation on the edge, *Proceedings of the AAAI Conference on Artificial Intelligence* 37 (2) (2023) 1339–1347. <https://doi.org/10.1609/aaai.v37i2.25232>.
- [18] Y. Kwon, W. Kim, H. Kim, HARD: Hardware-aware lightweight real-time semantic segmentation model deployable from edge to GPU, in: *Asian Conference on Computer Vision*, 2024, pp. 3552–3569. https://openaccess.thecvf.com/content/ACCV2024/html/Kwon_HARD__Hardware-Aware_lightweight_Real-time_semantic_segmentation_model_Deployable_from_ACCV_2024_paper.html.
- [19] D. Wang, B. Gong, L. Wang, On calibrating semantic segmentation models: Analyses and an algorithm, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 23652–23662.
- [20] P. de Jorge, R. Volpi, P.H.S. Torr, G. Rogez, Reliability in semantic segmentation: Are we on the right track?, in: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 2456–2465.
- [21] C. Sakaridis, D. Dai, L. Van Gool, ACDC: The adverse conditions dataset with correspondences for semantic driving scene understanding, in: *IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10765–10775.

- [22] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, B. Schiele, The Cityscapes dataset for semantic urban scene understanding, in: IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 3213–3223.
- [23] S. Xu, Y. Li, M. Lin, P. Gao, G. Guo, J. Lue, B. Zhang, EQ-Net: Elastic quantization neural networks, in: IEEE/CVF International Conference on Computer Vision, 2023, pp. 15072–15082.
- [24] Q. Jin, L. Yang, Z. Liao, AdaBits: Neural network quantization with adaptive bit-widths, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 2146–2156.